

Tiny TAC

Syntax And Semantics

Tiny TAC

Tiny TAC (`ttac`) is the small language used in the VCGen notes.

It keeps only the semantic core needed for examples:

- typed registers: `bool`, `int`, `bytemap`
- assignments, `havoc`, `assume`, `assert`
- basic blocks with explicit terminators
- phi nodes at joins

The goal is not to document `ctac` syntax. The goal is to make the VCGen algorithms precise.

Types And Expressions

Registers have one of three types:

$$\tau ::= \text{bool} \mid \text{int} \mid \text{bytemap}$$

The SMT model is intentionally direct:

$$\text{int} \mapsto \text{Int}$$

$$\text{bytemap} \mapsto \text{Int} \rightarrow \text{Int}$$

Core integer expressions:

$$i ::= n \mid x \mid i + i \mid i - i \mid i * i \mid \frac{i}{i}$$

Boolean expressions:

$$b ::= \text{true} \mid \text{false} \mid c \mid i < i \mid i \leq i \mid b \wedge b$$

ite is an expression operator, not a structured statement.

Maps

Loads and stores are expressions over bytemaps.

```
M := havoc  
i := havoc  
v := havoc  
x := M[i]  
M2 := M[i := v]
```

Semantically, M_2 agrees with M at every address except i , where it stores v .

Commands

Command forms:

```
x := e
x := havoc
x := phi [B1: x1, B2: x2]
assume b
assert c
```

`:=` is assignment. Equality is a separate expression/operator.

Example:

```
x := havoc
y := havoc
z := havoc
c := x < y
assume y < z
assert c
```

`assume` may use an expression. `assert` uses a named bool register.

Blocks And Control

Unstructured basic blocks. Branches are terminators.

```
entry:
  x := havoc
  y := havoc
  c := x < y
  if c goto ok else bad

ok:
  assert c
  goto exit

bad:
  assume not c
  halt

exit:
  halt
```

Phi Nodes

Phi nodes select by predecessor block.

```
L:
  x1 := 1
  goto J

R:
  xr := 2
  goto J

J:
  x := phi [L: x1, R: xr]
  goto exit
```

- Enter from L: $x := x_l$
- Enter from R: $x := x_r$

Semantic Contract

A ttac execution follows the expected operational semantics:

- assignment updates the destination register
- havoc chooses an arbitrary value of the destination type
- assume filters executions
- assert fails when its named condition is false
- terminators choose the next block

VC generation is bug-oriented:

$\text{VCGen}(P)$ is satisfiable $\Leftrightarrow P$ has an assertion-failure execution